

Sim. Eu incorporaria a lógica de sessões curtas do Analyst Gym, mas **não copiaria o produto**. O que vale aproveitar é a estrutura pedagógica: exercícios rápidos baseados em decisões reais, confiança declarada antes do feedback e formatos diferentes de sessão. O site atualmente apresenta quatro formatos — Lesson, Rapid Fire, Currency e Scenario — voltados respectivamente a raciocínio guiado, reconhecimento rápido de padrões, priorização de investigações e decisões sequenciais. 

Para o seu companion, eu adicionaria esses formatos **mais sessões especificamente desenhadas para arquitetura analítica e implementação em SQL**.

Abaixo está uma especificação que você pode entregar ao Claude Code praticamente como PRD.

## PRD — Analytical Reasoning Gym

### 1. Visão do produto

Construir uma aplicação web **mobile-first** para prática deliberada de raciocínio analítico, definição de métricas, arquitetura de consultas e SQL.

O produto não deve funcionar como um simples banco de exercícios de SQL.

O objetivo principal é desenvolver a capacidade de transformar:

```
Problema de negócio
    ↓
Pergunta analítica
    ↓
Definição das métricas
    ↓
População e granularidade
    ↓
Estrutura dos dados
    ↓
Plano de transformação
    ↓
SQL
    ↓
Validação
    ↓
Interpretação
```

A aplicação deve avaliar essas etapas separadamente.

O usuário pode saber perfeitamente a sintaxe de uma função e ainda assim construir uma análise incorreta. Portanto, **acertar o SQL não significa necessariamente acertar o exercício.**

---

## 2. Princípio central

Antes de permitir a implementação, o sistema deve frequentemente perguntar:

O que você está tentando medir?

Qual é a população?

O que uma linha do resultado representa?

Qual é o numerador?

Qual é o denominador?

Qual período está sendo considerado?

Quais dados são necessários?

Qual deve ser a sequência de transformações?

A aplicação deve ensinar:

**Concept first. Query second.**

---

## 3. Plataforma

Construir como **Progressive Web App (PWA)** responsiva.

Prioridade:

1. Celular
2. Desktop
3. Tablet

Deve ser possível:

- acessar pelo navegador;
- instalar na home screen do celular;
- continuar uma sessão iniciada no computador;
- fazer exercícios conceituais confortavelmente pelo celular;

- escrever SQL no celular quando desejado;
- usar teclado físico normalmente no desktop.

Não criar dependência de aplicativo nativo inicialmente.

---

## 4. Stack sugerida

Preferência por uma arquitetura simples:

```
Frontend
Next.js
TypeScript
Tailwind CSS

Backend
Next.js / API routes

Database
PostgreSQL

ORM
Drizzle ou Prisma

Authentication
Auth.js / equivalente

SQL execution sandbox
DuckDB-WASM para exercícios locais
ou
PostgreSQL isolado para exercícios server-side

PWA
Service Worker + manifest

Charts
Recharts

Editor SQL
CodeMirror 6
```

O banco de exercícios deve ser separado do banco operacional da aplicação.

---

## 5. Home

A home deve ser extremamente simples.

Exemplo:

---

🔥 12 day streak

Good afternoon.

Ready for today's workout?

[ DAILY SESSION — 8 min ]

---

Quick Practice

[ 2 min ] Rapid Fire

[ 5 min ] Metric Lab

[ 10 min ] Architecture

[ 15 min ] SQL Case

---

Weakest skills

|                    |     |
|--------------------|-----|
| Metric Definition  | 64% |
| Query Architecture | 58% |
| Window Functions   | 76% |

[ Practice weaknesses ]

---

O objetivo é abrir o celular e começar uma sessão em poucos segundos.

---

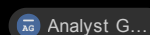
## 6. Tipos de sessão

### A. Daily Drill

Duração:

**2–5 minutos.**

Inspirado na ideia de julgamento analítico rápido do Analyst Gym.



Exemplo:

Weekly conversion dropped from 18% to 13%.

Traffic remained stable.

A checkout update was released Wednesday.

What would you investigate first?

Usuário escolhe uma hipótese.

Depois informa:

Confidence

- ☐ 20%
- ☐ 40%
- ☐ 60%
- ☐ 80%
- ☐ 100%

Feedback avalia:

- decisão;
- justificativa;
- confiança;
- calibração.

---

## 7. Metric Lab

Este deve ser um dos principais modos.

**Não existe SQL inicialmente.**

Exemplo:

Uma plataforma quer acompanhar a taxa de cancelamento de pedidos.

Perguntas progressivas:

### Etapa 1 — população

Quais pedidos entram no denominador?

### Etapa 2 — evento

O que exatamente conta como cancelamento?

### Etapa 3 — tempo

- O mês deve representar:
- o mês da criação
  - o mês do cancelamento
  - o depende da pergunta

### Etapa 4 — fórmula

Usuário monta:

Numerador:

[ \_\_\_\_\_ ]

Denominador:

[ \_\_\_\_\_ ]

### Etapa 5 — SQL

Somente agora:

Implemente sua definição.

## 8. Granularity Trainer

Modo específico para treinar granularidade.

Mostrar um problema:

Para cada loja e mês, calcule receita, participação na receita mensal da região e receita acumulada da loja.

Perguntar:

### Dataset original

1 linha = ?

Resposta:

pedido

Depois:

Qual granularidade precisamos primeiro?

Usuário seleciona:

- o pedido
- o loja
- loja × mês
- o região × mês

Depois:

E para calcular participação regional?

O usuário deve construir visualmente:

```
orders
1 row = order

      ↓ GROUP BY

store_month
1 row = store × month

      ↓ WINDOW

store_month_metrics
1 row = store × month
```

Só depois escreve SQL.

---

## 9. Query Architecture

Esse deve ser o modo mais importante da aplicação.

O usuário recebe um problema relativamente complexo, mas **não recebe editor SQL imediatamente**.

Exemplo:

Identifique produtos cuja receita cresceu mais de 30% em relação ao mês anterior e cuja participação na categoria também aumentou.

Interface:

STEP 1

Final grain

[ product × month ]

STEP 2

Required metrics

[ monthly product revenue ]

[ monthly category revenue ]

[ product share ]

[ previous month revenue ]

[ previous month share ]

STEP 3

Transformation plan

Usuário monta blocos:

RAW TRANSACTIONS

↓

[ Aggregate ]

↓

PRODUCT × MONTH

↓

[ Window ]

↓

PREVIOUS MONTH

↓

[ Calculate ]

↓

GROWTH + SHARE CHANGE

Depois:

**Implement this plan.**

---

## 10. Architecture Puzzle

Versão rápida do modo anterior.

Duração: 2–4 minutos.

Mostrar blocos embaralhados:

```
Apply LAG
Filter period
Aggregate transactions
Calculate growth
Join customers
```

Usuário precisa ordenar:

```
1.
2.
3.
4.
5.
```

Depois recebe explicação.

Excelente para celular porque não exige digitar SQL.

---

## 11. Rapid Fire

Inspirado no formato Rapid Fire do Analyst Gym: várias decisões curtas, avaliadas por precisão. 

12 questões consecutivas.

Exemplos:

1

Você precisa preservar todos os clientes, inclusive aqueles sem pedidos.

```
INNER JOIN
LEFT JOIN
CROSS JOIN
```

2

Uma linha atualmente representa uma transação.

Você precisa calcular receita mensal por empresa antes de comparar empresas.

```
GROUP BY primeiro  
WINDOW primeiro
```

3

Quer comparar uma observação com a anterior.

```
LAG  
LEAD  
RANK
```

4

Sua query passou de 100 mil para 4 milhões de linhas depois de um JOIN.

Primeira suspeita?

```
1:N / N:N join
```

Mas não limitar Rapid Fire à sintaxe.

Incluir perguntas como:

A taxa caiu de 20% para 10%. Isso significa queda de:

10%  
10 p.p.  
50%  
100%

## 12. Investigation Budget

Adaptação da ideia de **Currency**, em que o usuário possui recursos limitados e precisa escolher quais investigações valem o custo.  Analyst G...

Usuário começa com:

```
Investigation Budget: 100 credits
```

Problema:

Revenue caiu 18% esta semana.

Investigações disponíveis:

|                              |     |
|------------------------------|-----|
| Check traffic                | -10 |
| Check conversion             | -10 |
| Break down by country        | -15 |
| Break down by device         | -15 |
| Inspect individual customers | -30 |
| Check tracking changes       | -10 |
| Run statistical model        | -40 |
| Interview sales team         | -20 |

Cada investigação revela informação.

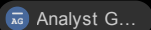
Objetivo:

**chegar a uma recomendação gastando o mínimo possível.**

Avaliar:

Correctness  
Investigation efficiency  
Prioritization  
Stopping judgment

## 13. Scenario

Inspirado no formato sequencial do Analyst Gym, no qual novas informações aparecem conforme decisões são tomadas. 

Exemplo:

### Segunda-feira

CEO pergunta:

Why did revenue fall yesterday?

Usuário decide como investigar.

Resultado aparece.

### Terça-feira

Descobre-se que mobile caiu.

Novo stakeholder pergunta:

Should we rollback the new app?

Usuário responde.

## Quarta-feira

A análise mostra que apenas novos usuários foram afetados.

Cada decisão modifica:

```
Analytical confidence
Stakeholder trust
Investigation cost
Decision quality
```

---

## 14. SQL Build

Modo tradicional de SQL, mas contextualizado.

Categorias:

```
Filtering
Aggregation
Joins
Dates
CTEs
Subqueries
Conditional aggregation
Window functions
Ranking
Running totals
Rolling windows
LAG / LEAD
Multi-grain analysis
Complex pipelines
```

Não apresentar:

“Faça um exercício de LAG.”

Apresentar:

“Identifique clientes cuja quantidade de pedidos aumentou significativamente em relação ao mês anterior.”

O usuário deve descobrir que `LAG` é apropriado.

---

## 15. Multi-Grain Challenge

Criar uma categoria inteira para cálculos que envolvem múltiplas granularidades.

Exemplos:

```
company × month  
versus  
month
```

```
product × store  
versus  
store
```

```
customer × week  
versus  
customer history
```

```
driver × day  
versus  
city × day
```

```
campaign × channel × month  
versus  
channel × month
```

O sistema deve ensinar explicitamente:

**Uma transformação pode precisar criar a granularidade necessária para a próxima transformação.**

---

## 16. Metric Debugger

Mostrar uma query que roda sem erros, mas produz uma métrica errada.

Exemplo:

```
SQL
```

```
SELECT
    month,
    SUM(cancelled) / COUNT(*) AS cancellation_rate
FROM orders
GROUP BY month;
```

Problema escondido:

alguns status não são elegíveis para o denominador.

Pergunta:

The SQL runs successfully. Is the metric correct?

O usuário precisa encontrar o problema **conceitual**, não sintático.

---

## 17. Query Debugger

Agora o inverso.

A lógica está correta, mas SQL está quebrado.

Tipos de erro:

```
Binder error
Parser error
Wrong alias
Invalid GROUP BY
Integer division
NULL behavior
Wrong JOIN key
WHERE destroying LEFT JOIN
Duplicated rows
Date type mismatch
Window nesting
```

Usuário recebe o erro e precisa investigar.

---

## 18. Grain Debugger

Mostrar:

```
Before JOIN:
100,000 rows
```

After JOIN:  
2,400,000 rows

Perguntar:

What would you check first?

Depois mostrar schemas.

Objetivo: detectar relações 1:N/N:N.

## 19. Metric Reverse Engineering

Mostrar somente o resultado:

| month | users  | converted | rate |
|-------|--------|-----------|------|
| Jan   | 10,000 | 1,500     | 15%  |
| Feb   | 11,000 | 1,430     | 13%  |

Perguntar:

What assumptions must be true for this metric to be meaningful?

Usuário deve questionar:

- definição de user;
- elegibilidade;
- janela;
- cohort;
- evento de conversão;
- duplicidade.

## 20. Explain Before You Code

Modo específico de entrevista/live analysis.

Cronômetro:

Problem shown.

You have 90 seconds.

DO NOT WRITE SQL.

Usuário precisa preencher:

```
Final grain:
Population:
Metric:
Time window:
Tables:
Plan:
```

Só depois o editor desbloqueia.

---

## 21. Ambiguous Requirements

Exercícios deliberadamente incompletos.

Exemplo:

| Calculate monthly retention.

Nenhuma definição adicional.

O objetivo é **não sair codando**.

Usuário deve escolher perguntas:

```
What counts as retained?
What is the cohort definition?
What is the observation window?
What event represents activity?
```

Pontuar a qualidade das perguntas.

---

## 22. Validation Lab

Query aparentemente pronta.

Pergunta:

| Before presenting this result, what would you validate?

Possibilidades:

```
row counts
NULLs
duplicates
join cardinality
min/max dates
distribution
impossible values
denominator
sample records
reconciliation with known totals
```

Usuário escolhe os checks mais importantes.

---

## 23. Confidence Calibration

Em praticamente todos os exercícios:

```
How confident are you?
```

```
20 — 40 — 60 — 80 — 100
```

Guardar:

```
confidence
correctness
```

Criar posteriormente:

```
Calibration Score
```

Exemplo:

```
Accuracy:           76%
Average confidence:  89%

Pattern:
You tend to be overconfident
in metric-definition questions.
```

A declaração de confiança antes do feedback também é parte explícita da abordagem do Analyst Gym.  Analyst G...

---

## 24. Biblioteca extensa de cases

Criar inicialmente **pelo menos 300 exercícios**.

Distribuição:

| Área               | Cases |
|--------------------|-------|
| Métricas           | 50    |
| Granularidade      | 40    |
| Query Architecture | 50    |
| SQL                | 50    |
| Debugging          | 30    |
| Investigação       | 25    |
| Validação          | 20    |
| Ambiguidade        | 15    |
| Comunicação        | 10    |
| Cenários longos    | 10    |

## 25. Domínios

Evitar concentrar tudo em um único contexto.

Usar:

E-commerce  
Fintech  
SaaS  
Telecom  
Marketplace  
Delivery  
Mobility  
Streaming  
Gaming  
Advertising  
Retail  
Logistics  
Healthcare operations

Travel  
Subscription products  
Education  
Energy  
Insurance  
B2B sales  
Consumer apps

## 26. Banco de métricas

A aplicação deve possuir uma biblioteca grande de métricas para gerar exercícios.

### Growth

Revenue  
GMV  
Orders  
AOV  
ARPU  
Growth rate  
Market share  
Contribution

### Product

Activation  
Conversion  
Retention  
Churn  
DAU  
WAU  
MAU  
Stickiness  
Feature adoption  
Funnel conversion

### Marketing

CTR  
CPC  
CPA  
CAC  
ROAS  
Conversion  
Incrementality

## Operations

Delivery time  
Cancellation rate  
SLA  
Failure rate  
Utilization  
Throughput

## Finance

Approval rate  
Default rate  
Delinquency  
Loss rate  
Exposure  
Recovery rate

## Customer

Repeat purchase  
Lifetime value  
Support rate  
NPS response  
Refund rate

O sistema deve variar especialmente **denominadores e populações elegíveis**, porque é aí que muitos erros analíticos aparecem.

---

## 27. Progressão de dificuldade

Não usar apenas Easy / Medium / Hard.

Usar níveis de arquitetura:

L1 – Single metric  
L2 – Aggregation  
L3 – Join  
L4 – Multiple metrics  
L5 – Multiple grains  
L6 – Sequential transformations  
L7 – Ambiguous business problem  
L8 – Investigation  
L9 – Complex case  
L10 – Open-ended analysis

## 28. Skill Graph

Cada exercício possui tags internas.

Exemplo:

JSON

```
{
  "skills": [
    "metric_definition",
    "grain",
    "aggregation",
    "window_function",
    "denominator",
    "validation"
  ]
}
```

Dashboard:

ANALYTICAL REASONING

|                    |                        |    |
|--------------------|------------------------|----|
| Metric Definition  | <div><div></div></div> | 78 |
| Population         | <div><div></div></div> | 69 |
| Granularity        | <div><div></div></div> | 58 |
| Query Architecture | <div><div></div></div> | 56 |
| SQL Syntax         | <div><div></div></div> | 87 |
| Debugging          | <div><div></div></div> | 84 |
| Validation         | <div><div></div></div> | 67 |
| Interpretation     | <div><div></div></div> | 75 |

## 29. Adaptive Practice

O sistema deve selecionar exercícios com base nos erros anteriores.

Exemplo:

Last 20 sessions:

|                          |     |
|--------------------------|-----|
| Window syntax            | 88% |
| Multi-grain architecture | 54% |
| Metric denominator       | 57% |

Próxima sessão:

40% Multi-grain  
30% Metric definition  
15% Validation  
15% Mixed

Não repetir simplesmente a mesma questão.

Variar domínio e estrutura mantendo a habilidade subjacente.

---

## 30. Error taxonomy

Todo erro deve ser classificado.

SYNTAX

SQL inválido.

SEMANTIC

SQL executa, mas cálculo está errado.

METRIC

Métrica definida incorretamente.

GRAIN

Granularidade incorreta.

POPULATION

População errada.

DENOMINATOR

Denominador incorreto.

JOIN

Relacionamento incorreto.

TIME

Janela temporal incorreta.

LEAKAGE

Informação futura utilizada.

ARCHITECTURE

Etapas na ordem errada.

VALIDATION

Resultado não validado.

INTERPRETATION

Conclusão não suportada.

Isso é essencial.

## 31. Feedback

Nunca responder apenas:

Wrong.

Feedback:

Result: Incorrect

|              |   |
|--------------|---|
| SQL syntax   | ✓ |
| Population   | ✓ |
| Granularity  | ✗ |
| Metric       | ✓ |
| Architecture | ✗ |
| Validation   | — |

The issue happened before the window function.

Your current dataset still has one row per transaction,  
but the calculation requires one row per company × month.

Think:

```
transactions
  ↓
?
  ↓
company × month
  ↓
window
```

Não entregar imediatamente a query correta.

## 32. Hint system

Três níveis.

### Hint 1 — conceito

What should one row represent before calculating this metric?

### Hint 2 — arquitetura

```
transactions
  ↓
GROUP BY ?
  ↓
?
  ↓
WINDOW
```

### Hint 3 — implementação

Mostrar trecho parcial.

Cada hint reduz a pontuação de autonomia.

---

## 33. Autonomy Score

Criar uma métrica específica:

Autonomy Score

Considerar:

```
solved without hints
number of hints
wrong attempts
architecture revisions
time to solution
validation performed
```

Exemplo:

```
Correctness:    92
Autonomy:       68
```

Isso diferencia:

“conseguiu depois de três dicas”

de

“estruturou sozinho”.

---

## 34. Workout Mode

Sessões completas:

### 10 minutos

- 2 Rapid Fire
- 1 Metric Lab
- 1 Architecture Puzzle
- 1 SQL

### 20 minutos

- 5 Rapid Fire
- 2 Metrics
- 2 Architecture
- 1 SQL Case

### 45 minutos

- Warm-up
- Metric definition
- Architecture
- SQL implementation
- Debugging
- Open case
- Debrief

---

## 35. Boss Fight

Uma vez por semana.

Case de 30–45 minutos.

Exemplo:

Uma plataforma observou queda de receita nas últimas seis semanas. Determine o que está acontecendo e produza uma recomendação.

O sistema fornece:

```
customers
orders
products
payments
regions
campaigns
```

Nenhuma função SQL sugerida.

Usuário precisa decidir tudo.

No final:

```
Problem framing
Metric definition
Architecture
SQL
Validation
Interpretation
Recommendation
```

## 36. Mobile UX

No celular, evitar editor gigante.

Usar cards progressivos:

```
| What is the final grain? |
|                           |
| [ company × month       ] |
|                           |
|      Continue →        |
```

Architecture Builder pode ser drag-and-drop:

```
[ GROUP BY ]
[ JOIN ]
[ FILTER ]
```

```
[ WINDOW ]  
[ CALCULATE ]
```

Arrastar para:

```
RAW  
↓  
[      ]  
↓  
[      ]  
↓  
RESULT
```

Isso torna **arquitetura analítica** **praticável no ônibus, sofá ou fila**, sem precisar digitar queries grandes.

---

## 37. “Explain it”

Após resolver:

Explain your reasoning in 30 seconds.

Pode inicialmente ser texto.

Futuramente adicionar áudio/transcrição.

Avaliar presença de:

```
population  
grain  
metric definition  
comparison  
result  
limitation
```

## 38. Case Generator

Construir cases parametrizados.

Schema:

```
JSON  
  
{  
  "domain": "ecommerce",
```

```
"difficulty": 6,  
"skills": [  
  "grain",  
  "metric_definition",  
  "window_function"  
],  
"prompt": "...",  
"dataset": "...",  
"expected_reasoning": {},  
"expected_architecture": [],  
"solution_sql": "...",  
"common_errors": [],  
"hints": []  
}
```

Isso permite futuramente gerar milhares de variações.

---

## 39. Regra para conteúdo gerado por IA

A IA não deve simplesmente gerar uma questão e uma resposta.

Cada exercício gerado deve conter internamente:

```
Canonical metric definition  
Expected population  
Expected grain  
Expected transformation graph  
Expected output  
Reference SQL  
Validation checks  
Common wrong solutions  
Reasoning rubric
```

Executar automaticamente o SQL de referência antes de publicar o exercício.

Se o resultado não puder ser validado, o exercício não entra no banco.

---

## 40. Seed dataset

Criar datasets sintéticos reutilizáveis.

Por exemplo:

**CommerceDB**

users  
sessions  
orders  
order\_items  
payments  
products  
merchants  
refunds

## SubscriptionDB

accounts  
subscriptions  
plans  
invoices  
payments  
usage\_events  
support\_tickets

## MobilityDB

users  
drivers  
rides  
payments  
cities  
promotions

## MarketingDB

campaigns  
impressions  
clicks  
conversions  
spend  
customers

## RetailDB

stores  
customers  
transactions  
transaction\_items  
products  
inventory

Cada banco deve ter relações 1:N e algumas N:N para permitir armadilhas realistas.

---

## 41. Cases que PRECISAM existir

A biblioteca inicial deve obrigatoriamente incluir vários exercícios de:

GROUP BY → WINDOW

JOIN → GROUP BY → WINDOW

GROUP BY → LAG

GROUP BY → WINDOW → FILTER

multiple CTEs with different grains

company vs total

segment vs overall population

current period vs historical period

rolling average

running total

share of total

percentage-point change

relative percentage change

cohort metrics

conditional denominators

eligible populations

deduplication before aggregation

1:N join inflation

N:N join inflation

LEFT JOIN filter mistake

NULL denominator

integer division

date cohort vs event date

current state vs historical state

latest record per entity

first event per entity

event sequences

multiple events in rolling window

---

## 42. Métrica principal do produto

Não otimizar para:

Quantos exercícios o usuário fez?

O objetivo é:

Quanto menos ajuda ele precisa para estruturar problemas novos?

Dashboard principal:

30 DAY DEVELOPMENT

Correctness

71 → 84

Autonomy

54 → 73

Architecture

49 → 76

Metric Definition

63 → 81

SQL

82 → 88

---

## 43. MVP

Eu não construiria tudo de uma vez.

## V1

Implementar:

Authentication  
PWA/mobile layout  
Daily Drill  
Metric Lab  
Granularity Trainer  
Architecture Puzzle  
SQL Build  
Feedback  
Hints  
Skill tracking  
Streak

Criar **100 exercícios manuais/seeded** de alta qualidade.

## V2

Adicionar:

Rapid Fire  
Investigation Budget  
Scenario  
Adaptive practice  
Autonomy Score  
Boss Fight

## V3

Adicionar:

AI case generation  
Personalized workouts  
Voice reasoning  
Advanced analytics  
Large case datasets

---

## 44. Critério de sucesso

Depois de algumas semanas utilizando o produto, diante de:

“Calcule a taxa X de cada segmento, compare-a com a taxa geral do mês e mostre sua evolução acumulada.”

o comportamento esperado não é abrir o editor imediatamente.

Deve ser automático pensar:

```
O que é X?  
↓  
Qual população?  
↓  
Qual numerador/denominador?  
↓  
Resultado final em qual grain?  
↓  
Preciso criar qual grain intermediário?  
↓  
Quais etapas dependem das anteriores?  
↓  
GROUP BY?  
↓  
WINDOW?  
↓  
Agora SQL.
```

**Essa mudança de comportamento é o verdadeiro produto.**

Eu usaria do Analyst Gym principalmente a filosofia de sessões curtas, variedade de formatos, prática de julgamento e calibração de confiança; e adicionaria por cima uma camada muito mais forte de **metric definition** → **grain** → **query architecture** → **implementation**, que é o diferencial deste companion.  Analyst G...

Referência: Analyst Gym — Session Types